

아래는 보고서에 그대로 붙여넣을 수 있는 [Make vs Zapier](#) 비교 분석 문장형 보고서입니다.

이번 비교는 “**Google Form 제출 → 점수 기준 조건 분기 → Google Sheets 기록 + Slack/Discord 알림**”이라는 동일한 자동화 과제를 두 도구에 적용했을 때의 차이를 중심으로 정리하였다.

[Make vs Zapier](#) 비교 분석

본 비교 분석은 동일한 업무 시나리오인 “Google Form 제출 → 점수 기준 조건 분기 → Google Sheets 기록 → Slack 또는 Discord 알림 전송”을 기준으로, [Make](#)와 [Zapier](#)의 구현 방식, 사용성, 조건 분기 구조, 실행 방식, 요금제 제약, 그리고 적합한 활용 상황을 비교한 것이다. 두 도구 모두 노코드 자동화 플랫폼으로서 트리거와 액션을 연결해 반복 업무를 자동화할 수 있지만, 워크플로우를 설계하고 표현하는 방식에서 분명한 차이를 보인다. [Source](#) [Source](#)

가장 먼저 확인할 수 있는 차이는 **워크플로우 표현 방식**이다. Make는 시나리오를 시각적 캔버스 위에 모듈 단위로 배치하는 구조를 사용하며, 각 모듈 사이의 흐름을 선으로 연결해 전체 로직을 한눈에 볼 수 있도록 설계되어 있다. 특히 Make의 Router는 하나의 입력 흐름을 여러 경로로 나누고, 각 경로에 필터를 설정하여 서로 다른 처리를 수행할 수 있게 한다. 또한 fallback route를 둘 수 있어 어떤 조건에도 맞지 않는 데이터를 별도로 처리하는 구조도 가능하다. 이러한 시각적 구조는 자동화 흐름 자체를 설명하거나 발표해야 하는 교육·과제 환경에서 큰 장점이 된다. [Source](#)

반면 Zapier는 **단계형 리스트 구조**를 중심으로 워크플로우를 구성한다. 각 단계가 위에서 아래로 순서대로 쌓이는 형태이기 때문에, 처음 자동화를 접하는 사용자 입장에서는 구조를 이해하기 쉽고 설정 과정도 비교적 직관적이다. 다만 분기가 많아질수록 전체 흐름이 시각적으로 넓게 펼쳐지기보다 단계 내부에 숨겨지는 경향이 있어, 복잡한 분기 구조를 한눈에 해석하는 데에는 Make보다 불리할 수 있다. 그럼에도 불구하고 Zapier의 Paths는 “If A happens, do X / If B happens, do Y”와 같은 전형적인 조건 분기 로직을 손쉽게 구현할 수 있도록 해 주며, 실무에서 자주 쓰이는 승인/반려, 유형 분류, 결과별 알림 같은 시나리오에 매우 적합하다. [Source](#)

이번 프로젝트와 같이 **Google Form 응답을 트리거로 사용하는 상황**에서는 실행 방식에서도 차이가 나타난다. Zapier의 Google Forms 앱은 New Form Response (Instant) 트리거를 제공하므로, 새로운 응답이 제출되면 Zap이 즉시 시작되는 구조를 만들 수 있다. 반면 Make에서는 실습 환경에서 Google Form 응답이 저장된 Google Sheets를 Watch New Rows 로 감시하는 방식을 사용하는 경우가 많으며, 이 방식은 새 행이 추가된 뒤 Make가 이를 감지하는 구조다. Make의 기본 스케줄 설정은 15분 간격이며, 실행 주기는 플랜에 따라 조정된다. 따라서 같은 “폼 제출 자동화”라도 Zapier는 보다 직접적인 즉시 반응 구조를 만들기 쉽고, Make는 시트 기반 감시와 시나리오 스케줄에 의존하는 형태가 되기 쉽다. [Source](#) [Source](#) [Source](#)

조건 분기 구조를 비교하면, 두 도구 모두 분기형 자동화를 지원하지만 설계 철학은 다소 다르다. Make의 Router는 각 경로를 시각적으로 나눠 보여 주며, 필터를 각 라우트의 입구에 명확하게 배치한다. 문

서에서도 Router는 데이터를 여러 체인으로 나누고, 각 경로를 조건에 따라 다르게 처리하며, 경로는 병렬이 아니라 순차적으로 처리된다고 설명한다. 이는 사용자가 “어떤 입력이 어느 경로로 흘러가는지”를 시각적으로 이해하기 쉽게 해 준다. [Source](#)

Zapier의 Paths 역시 순차 실행 방식이며, 경로는 왼쪽에서 오른쪽 순서로 평가 및 실행된다. 다만 Zapier는 분기 로직을 “경로별 규칙” 중심으로 보여 주기 때문에, 논리 자체는 깔끔하지만 시각적 흐름도라는 느낌은 Make보다 약하다. 대신 조건 설정 방식은 매우 명료하다. 사용자는 필드, 조건, 값을 차례로 선택해 경로 규칙을 만들 수 있고, 필요하면 Custom rules, Always run, Fallback 같은 옵션도 사용할 수 있다. 따라서 Zapier는 “조건 규칙을 빠르게 설정하고 바로 액션을 붙이는” 데 강점이 있으며, Make는 “분기 구조 자체를 설계하고 설명하는” 데 더 유리하다고 볼 수 있다. [Source](#) [Source](#)

사용 난이도와 학습 곡선 측면에서도 차이가 존재한다. Zapier는 비교적 단순한 UI와 단계형 편집기를 사용하기 때문에 초보자가 처음 접했을 때 진입 장벽이 낮다. 예를 들어 Google Forms에서 응답을 받아 Google Sheets에 기록하고 Slack에 메시지를 보내는 작업은 “트리거 1개 + 액션 몇 개” 방식으로 자연스럽게 이어진다. 반면 Make는 자유도가 높은 대신, 모듈 연결 구조와 데이터 흐름, 라우터 분기, 필터 위치 등을 함께 이해해야 하기 때문에 처음에는 다소 복잡하게 느껴질 수 있다. 그러나 일단 구조를 이해하면, 복잡한 로직을 더 정교하고 시각적으로 설계할 수 있다는 점에서 중급 이상 사용자에게는 Make가 더 큰 설계 자유도를 제공한다. [Source](#) [Source](#) [Source](#)

Google Sheets와의 연동 방식에서도 실무적인 차이를 볼 수 있다. Make의 Watch New Rows 는 시트에 새 행이 생기면 이를 감지하는 방식인데, 문서상 시트에 중간 빈 행이 있으면 이후 행을 처리하지 않을 수 있다는 주의사항이 있다. 즉, 시트 구조를 일정하게 유지하는 관리가 필요하다. 반면 Zapier의 Google Sheets 앱은 Create Spreadsheet Row , New Spreadsheet Row , New or Updated Spreadsheet Row 같은 비교적 표준적인 행 단위 작업을 안정적으로 제공하며, 결과 기록용 시트를 분리해 쓰는 구조와 잘 맞는다. 따라서 시트 기반 자동화의 **정교한 구조 설계**에는 Make가, **빠른 표준 업무 연결**에는 Zapier가 더 간편한 편이라고 평가할 수 있다. [Source](#) [Source](#)

요금제와 기능 접근성은 실제 도구 선택에서 매우 중요한 요소다. Zapier의 가격 정책은 task 기반이며, Free 플랜은 월 100 tasks, 무제한 Zap/Tables/Forms, 그리고 two-step Zap workflows를 포함한다. 즉, 단순 자동화에는 무료 플랜이 유용하지만, 이번 과제처럼 Trigger 이후 Paths와 다수의 액션이 이어지는 구조는 일반적으로 유료 플랜에서 더 현실적으로 구현된다. 또한 Paths와 Filters 같은 조건 로직 기능은 도움말에서도 유료 플랜 맥락에서 안내되는 경우가 많다. 다만 Zapier는 Path by Zapier 같은 내장 데이터 도구가 task 계산에서 제외될 수 있다는 장점도 있다. [Source](#) [Source](#) [Source](#)

Make는 실행량과 기능 한도를 크레딧 및 플랜 기반으로 관리하며, 시나리오 스케줄 간격, 파일 크기, 실행 시간, 동시 실행 등 운영 관련 제약이 플랜에 따라 달라진다. 따라서 Make는 단순히 “몇 단계까지 무료인가”보다, **얼마나 자주 돌릴 것인지, 얼마나 복잡한 시나리오인지, 얼마나 많은 데이터가 오가는지**를 기준으로 판단하는 쪽이 더 적절하다. 즉, Zapier는 task 중심 비용 구조가 명확하고, Make는 운영 자원과 실행 성격에 따라 체감 비용 구조가 달라지는 편이라고 정리할 수 있다. [Source](#) [Source](#)

메신저 연동 측면에서는 두 도구 모두 Slack과 Discord를 지원하지만, 설정 중 자주 마주치는 문제는 조금 다르다. Make의 Slack 모듈은 bot 연결을 사용할 경우 봇이 대상 채널 멤버여야 하며, 그렇지 않으면 오류가 발생할 수 있다. Zapier의 Slack 연동 역시 channel_not_found 오류가 자주 발생하는데, 이

경우 연결된 계정이 해당 채널 멤버가 아니거나 잘못된 워크스페이스를 선택했을 가능성이 크다. Discord의 경우 Zapier는 Send Channel Message 를 지원하지만, 서버 권한과 Manage Webhooks 권한이 필요하다. 따라서 두 도구 모두 메신저 연동 자체는 가능하지만, 실제 운영 전에는 채널 접근 권한과 연결 계정을 반드시 점검해야 한다. [Source Source Source](#)

이번 과제 기준으로 보면, **Make는 교육용 설명과 시각적 설계에 강하고, Zapier는 빠른 구현과 즉시성에 강하다**고 정리할 수 있다. Make는 Router 기반 구조 덕분에 “입력이 어떤 조건을 거쳐 어느 결과로 가는지”를 화면으로 설명하기 좋고, 과제 발표나 보고서 캡처에도 유리하다. 반면 Zapier는 Google Forms의 즉시 트리거와 간단한 단계형 편집 방식 덕분에 실제 업무에서 빠르게 자동화를 시도하고 검증하는 데 적합하다. 즉, 워크플로우를 구조적으로 보여 주는 것이 중요하면 Make가 유리하고, 빠르게 동작하는 실무형 자동화를 만드는 것이 중요하면 Zapier가 유리하다고 볼 수 있

다. [Source Source Source](#)

최종적으로, 본 과제의 제출 목적을 기준으로 Make와 Zapier 모두 충분히 적합하지만, 선택 기준은 분명하다. 자동화 구조를 **시각적으로 설계하고 비교 분석까지 함께 제시해야 하는 학습·과제 상황**에서는 Make가 더 설득력 있는 도구이며, **실제 서비스처럼 품 제출 직후 바로 반응하는 자동화 경험**을 강조하고 싶다면 Zapier가 더 적합하다. 따라서 두 도구는 단순히 어느 하나가 더 우수하다고 보기보다, 사용 목적과 강조하고 싶은 자동화 특성에 따라 강점이 달라지는 상호보완적 플랫폼으로 이해하는 것이 타당하다.

노코드 자동화 도구 비교 및 구현 보고서

[Make](#)와 [Zapier](#)를 활용한 Google Form 기반 조건 분기 자동화

본 보고서는 동일한 업무 시나리오인 “Google Form 제출 → 점수 기준 조건 분기 → Google Sheets 기록 → Slack 또는 Discord 알림 전송”을 두 개의 대표적인 노코드 자동화 도구인 [Make](#)와 [Zapier](#)에서 각각 구현하고, 그 절차와 특징을 비교 분석한 결과를 정리한 것이다. 두 도구 모두 트리거와 액션을 연결하여 반복 업무를 자동화할 수 있지만, 워크플로우를 구성하는 방식과 실행 구조, 분기 설계 방식, 요금제 제약에서 차이를 보인다. [Source Source](#)

본 과제에서 구현한 자동화의 핵심 목적은 사용자가 Google Form에 입력한 점수 값을 기준으로 자동 판정을 수행하고, 그 결과를 별도의 Google Sheets에 기록하며, 동시에 담당자에게 메신저로 알림을 보내는 것이다. 이를 통해 수작업으로 응답 내용을 확인하고 판정하며 메시지를 전달하던 반복 업무를 자동화할 수 있다. 조건 분기, 기록, 알림의 세 요소가 모두 포함되므로, 트리거·액션·조건 분기를 요구하는 노코드 자동화 학습 과제로 적합하다. [Source Source](#)

전체 자동화 시나리오 개요

전체 시나리오는 동일하게 설계하였다. 먼저 [Google Forms](#)에서 이름, 이메일, 점수 항목을 가진 입력 폼을 구성하고, 이 응답을 [Google Sheets](#)에 저장하도록 연결하였다. 이후 자동화 도구가 새 응답을 감지하면 점수를 기준으로 합격과 불합격을 분기하고, 결과를 별도의 결과 시트에 누적 기록한 뒤, Slack 또는 Discord 채널로 결과 메시지를 전송하도록 하였다. Google Forms 응답은 연결된 Google Sheets에 저장할 수 있으며, 이 구조는 두 플랫폼 모두와 연계하기에 적합하다. [Source](#) [Source](#)

결과 기록용 시트는 원본 응답 시트와 분리하여 구성하였다. 원본 응답 시트는 Google Form이 자동으로 채우는 입력 데이터 영역으로 사용하고, 결과 시트는 자동화가 처리한 PASS/FAIL 판정과 알림 채널, 비고 등을 저장하는 출력 데이터 영역으로 사용하였다. 이렇게 입력과 출력을 분리하면 원본 데이터 보존이 쉬워지고, 자동화 결과를 명확히 관리할 수 있으며, 시나리오 검증과 보고서 작성에도 유리하다. [Source](#) [Source](#)

[Make](#) 구현 절차

[Make](#)에서는 Google Form을 직접 트리거로 사용하기보다, Google Form 응답이 저장되는 Google Sheets를 감시하는 방식으로 구현하였다. Google Forms 응답은 Google Sheets에 자동 저장할 수 있고, Make의 Google Sheets Watch New Rows 모듈은 지정한 시트에 새 행이 추가될 때 이를 감지하는 트리거 역할을 한다. 이 방식은 과제 실습 환경에서 구현이 안정적이고, 결과 검증과 흐름 설명이 쉬운 장점이 있다. [Source](#) [Source](#)

구현을 위해 먼저 Google Form을 생성하고 이름, 이메일, 점수 항목을 추가하였다. 점수는 이후 조건 분기의 기준이 되므로 숫자형으로 입력되도록 설정하였다. 이후 Google Form의 Responses 메뉴에서 응답 저장용 Google Sheets를 연결하여, 응답이 들어올 때마다 새로운 행이 자동 생성되도록 하였다. 별도로 Processed_Results 시트를 만들어 제출 시각, 이름, 이메일, 점수, 결과, 알림 채널, 비고 컬럼을 미리 정의하였다. [Source](#)

다음으로 Make에서 새 시나리오를 생성하고, 첫 번째 모듈로 [Google Sheets](#) Watch New Rows 를 추가하였다. 이 모듈은 원본 응답 시트에 새 행이 생길 때마다 데이터를 읽어온다. Google Sheets 연결은 Create a connection 을 통해 Google 계정을 인증하는 방식으로 이루어지며, 대상 스프레드시트와 시트를 선택한 뒤 테스트 응답을 제출하여 입력 데이터가 정상적으로 들어오는지 확인하였다. Make 문서에 따르면 시트에 중간 빈 행이 있으면 이후 행이 처리되지 않을 수 있으므로, 원본 응답 시트는 빈 줄 없이 연속적으로 유지해야 한다. [Source](#) [Source](#)

입력 데이터 확인이 끝난 뒤에는 [Router](#) 모듈을 추가하여 조건 분기를 구성하였다. Router는 하나의 입력 흐름을 여러 경로로 나누고, 각 경로마다 서로 다른 조건과 액션을 적용할 수 있게 하는 기능이다. 본 프로젝트에서는 두 개의 라우트를 만들었으며, 첫 번째 라우트는 $Score \geq 80$ 조건을 가진 합격 경로로, 두 번째 라우트는 $Score < 80$ 조건을 가진 불합격 경로로 설정하였다. Make 문서에 따르면 Router의 각 경로는 순차적으로 처리되며, 필요할 경우 fallback route도 설정할 수 있다. [Source](#)

합격 경로에는 먼저 Google Sheets의 Add a Row 모듈을 추가하여 결과 시트에 PASS 판정 결과를 기록하도록 구성하였다. 이때 제출 시각, 이름, 이메일, 점수는 원본 응답 시트에서 가져오고, 결과 컬럼

에는 PASS, 알림 채널에는 Slack 또는 Discord, 비교에는 점수 80 이상을 기록하였다. 그 다음 단계로 Slack의 Send a Message 또는 Discord의 Send a message 모듈을 추가하여, 합격 결과를 메신저 채널로 전송하도록 설정하였다. Slack의 경우 Make 문서상 봇이 대상 채널 멤버가 아니면 오류가 발생할 수 있으므로, 채널 참여 여부를 사전에 확인하였다. Discord는 채널, 스레드, DM 대상으로 메시지 전송을 지원한다. [Source](#) [Source](#) [Source](#)

불합격 경로도 동일한 구조로 구성하였다. 먼저 Add a Row 모듈로 결과 시트에 FAIL 결과를 기록하였고, 비교에는 점수 80 미만을 입력하였다. 이후 Slack 또는 Discord 메시지 모듈을 사용하여 불합격 알림 메시지를 전송하도록 하였다. 이렇게 함으로써 같은 입력 데이터 구조가 점수값에 따라 서로 다른 라우트로 나뉘어 각기 다른 결과 기록과 알림 전송을 수행하도록 구현하였다. [Source](#) [Source](#) [Source](#)

구현 완료 후에는 실제 응답 데이터를 사용해 두 경로를 모두 테스트하였다. 점수 90점 응답을 제출했을 때는 합격 경로가 실행되어 결과 시트에 PASS가 기록되고 합격 알림이 전송되었으며, 점수 65점 응답을 제출했을 때는 불합격 경로가 실행되어 FAIL이 기록되고 불합격 메시지가 전송되었다. 이를 통해 Router 기반 조건 분기와 후속 액션이 정상적으로 동작함을 확인하였다. [Source](#)

Make의 시나리오는 스케줄에 따라 실행된다. 기본적으로 시나리오는 15분 간격으로 실행되며, 상단의 스케줄 설정에서 실행 주기를 조정할 수 있다. 따라서 Google Form 제출 직후 완전한 즉시 반응이라기 보다는, 연결된 Google Sheets의 새 행을 감지하는 구조에 가깝다. 이러한 특성은 실시간성보다는 시각적 설계와 구조 설명이 중요한 학습용 구현에서 오히려 이해하기 쉬운 장점으로 작용하였다. [Source](#) [Source](#)

Zapier 구현 절차

Zapier에서는 [Google Forms](#)의 New Form Response (Instant) 트리거를 직접 사용하였다. 이는 사용자가 폼을 제출하는 즉시 Zap 실행이 시작되는 구조이므로, Make의 Google Sheets 감시 방식보다 더 직접적인 이벤트 기반 자동화라고 볼 수 있다. 다만 Zapier 도움말은 실제 활용을 위해 Google Form 응답이 Google Sheets에 저장된 상태를 전제로 설명하고 있으므로, 응답 시트 연결은 동일하게 유지하였다. [Source](#)

먼저 Zapier에서 Google Forms, Google Sheets, Slack 또는 Discord 계정을 각각 연결하였다. Zapier의 앱 연결은 Apps → + Add connection → 앱 선택 → 로그인 및 권한 승인 구조로 이루어진다. 그 후 새 Zap을 만들고, 첫 단계의 Trigger 앱으로 Google Forms를 선택한 뒤 Trigger Event를 New Form Response로 설정하였다. 대상이 되는 폼을 선택하고 테스트를 수행하여, 최근 제출된 응답 데이터가 정상적으로 읽히는지 확인하였다. [Source](#) [Source](#)

트리거 다음 단계에서는 [Paths by Zapier](#)를 추가하였다. Paths는 조건별로 서로 다른 액션 체인을 실행하는 분기 기능으로, “If A happens, do X / If B happens, do Y”와 같은 논리를 구성할 수 있다. 본 프로젝트에서는 Path A와 Path B 두 개의 경로를 사용하였다. Path A는 Score >= 80 조건을 가진 합격 경로로, Path B는 Score < 80 조건을 가진 불합격 경로로 설정하였다. Zapier 문서에 따르면 Paths의 분기들은 동시에 실행되지 않고, 왼쪽에서 오른쪽 순서로 순차적으로 평가 및 실행된다. [Source](#)

합격 경로에는 먼저 Google Sheets의 `Create Spreadsheet Row` 액션을 추가하여 결과 기록 시트에 PASS 결과를 저장하도록 하였다. 제출 시각, 이름, 이메일, 점수는 트리거 단계에서 가져왔고, 결과 컬럼에는 PASS, 알림 채널에는 Slack 또는 Discord, 비교에는 점수 80 이상 을 기록하였다. 이후 Slack의 `Send Channel Message` 또는 Discord의 `Send Channel Message` 액션을 추가하여 합격 메시지를 전송하도록 구성하였다. Slack은 채널 메시지 전송을 지원하지만, 연결된 Slack 계정이 해당 채널에 접근 가능해야 하며, 그렇지 않으면 `channel_not_found` 오류가 발생할 수 있다. Discord는 채널 메시지 전송을 지원하지만, 서버 권한과 Manage Webhooks 권한이 필요하다.

다. [Source](#) [Source](#) [Source](#) [Source](#)

불합격 경로도 동일하게 구성하였다. `Create Spreadsheet Row` 액션으로 결과 시트에 FAIL 을 기록하고, Slack 또는 Discord의 메시지 전송 액션을 붙여 불합격 알림을 보내도록 하였다. 이렇게 하여 Zapier에서도 하나의 입력 데이터가 조건에 따라 두 개의 서로 다른 결과 경로로 분기되고, 각 경로에서 기록과 알림이 동시에 이루어지는 자동화를 완성하였다. [Source](#) [Source](#) [Source](#)

테스트는 합격과 불합격 두 경우를 각각 실행하여 검증하였다. 점수 90점 응답을 제출했을 때는 Path A 가 실행되어 PASS가 기록되고 합격 메시지가 전송되었으며, 점수 65점 응답을 제출했을 때는 Path B 가 실행되어 FAIL이 기록되고 불합격 메시지가 전송되었다. 이를 통해 Google Forms의 즉시 트리거와 Paths 기반 분기 구조가 정상적으로 동작함을 확인하였다. [Source](#)

Zapier 구현에서 주의할 점은 플랜 제약이다. 무료 플랜은 월 100 tasks와 two-step Zap workflows 중 심이므로, 이번처럼 분기와 여러 액션이 포함된 구조는 일반적으로 유료 플랜에서 구현하는 것이 적절하다. 또한 조건 분기를 위한 Paths와 Filters는 도움말에서도 유료 플랜 환경과 함께 설명되는 경우가 많다. 따라서 기능 구현 가능 여부와 단계 수 제한을 함께 고려해야 한다. [Source](#) [Source](#)

[Make](#)와 [Zapier](#) 비교 분석

두 도구의 가장 큰 차이는 워크플로우를 표현하는 방식이다. Make는 시나리오를 시각적 캔버스 위에 모듈과 연결선으로 표시하므로, 데이터가 어디서 들어와 어떤 경로를 거쳐 어디로 가는지 한눈에 볼 수 있다. 특히 Router와 필터 구조는 조건 분기 로직을 시각적으로 설명하기에 매우 적합하다. 반면 Zapier는 단계형 편집기를 사용하므로 처음 자동화를 접하는 사용자에게는 훨씬 직관적이고 접근성이 높지만, 분기가 많아질수록 전체 흐름을 한눈에 읽기에는 Make보다 불리할 수 있다. [Source](#) [Source](#)

트리거 방식에서도 차이가 있다. Zapier는 Google Forms의 `New Form Response (Instant)` 를 직접 지원하므로, 폼 제출 즉시 자동화가 시작되는 구조를 만들기 쉽다. 반면 Make는 이번 구현에서 Google Sheets의 `Watch New Rows` 를 사용하였고, 이는 폼 응답이 시트에 저장된 후 이를 감지하는 구조다. 따라서 실시간 반응성과 직접성은 Zapier가 더 강하다고 볼 수 있다. 반대로 Make는 즉시성보다는 워크플로우 구조를 명확히 보여 주는 데 장점이 있었다. [Source](#) [Source](#) [Source](#)

조건 분기 기능은 두 도구 모두 충분히 강력했지만 사용 감각은 다르다. Make의 Router는 각 경로를 독립된 라인으로 보여 주며, 경로 입구에 필터를 걸어 시각적으로 분기를 표현한다. Zapier의 Paths는 경로별 규칙을 정의하고 그 아래에 액션을 쌓는 방식으로, 논리 구성은 매우 명확하지만 구조적 표현은 상

대적으로 단순하다. 따라서 발표나 시각적 설명이 중요한 학습 상황에서는 Make가, 빠른 규칙 설정과 실무형 연결에서는 Zapier가 더 적합하다고 판단된다. [Source Source](#)

사용 난이도 측면에서는 Zapier가 초보자 친화적이었다. Google Forms 트리거를 직접 연결하고, 단계별로 액션을 추가하는 구조는 직관적이며 빠르게 자동화를 만들 수 있다. 반면 Make는 모듈, 라우터, 필터, 시나리오 스케줄 등 함께 이해해야 할 요소가 많아 처음에는 다소 복잡하게 느껴질 수 있다. 그러나 복잡한 흐름을 정교하게 설계하고 시각적으로 제시하는 자유도는 Make가 더 높았다. [Source Source](#)

요금제와 기능 접근성도 중요한 차이였다. Zapier는 task 기반 과금 구조와 무료 플랜의 단계 제한이 명확하여, 단순 자동화는 무료로 체험하기 쉽지만 분기와 다단계 액션이 필요한 구조는 빠르게 유료 플랜 요구로 이어진다. Make는 실행 빈도, 실행 시간, 파일 크기, 동시 실행 등 운영 단위 제약이 플랜에 반영되므로, 비용 체감 방식이 조금 더 구조적이다. 즉, Zapier는 “얼마나 많은 작업이 실행되는가” 중심으로, Make는 “얼마나 복잡하고 자주 실행되는 시나리오인가” 중심으로 판단하는 경향이 강하다. [Source Source](#)

메신저 연동에서는 두 도구 모두 충분히 활용 가능했지만, 사전 권한 설정이 중요했다. Make의 Slack은 봇이 채널 멤버여야 하고, Zapier의 Slack은 계정 또는 채널 접근권이 맞지 않으면 `channel_not_found` 오류가 발생할 수 있다. Discord는 Zapier에서 메시지 전송이 가능하지만, 서버 관리 권한과 웹훅 권한이 필요하다. 따라서 실제 업무에 적용할 때는 자동화 로직 자체보다도 연결 권한과 채널 접근 설정을 먼저 점검해야 했다. [Source Source Source](#)

종합적으로 평가하면, Make는 시각적 설계와 교육용 설명에 강하고, Zapier는 빠른 연결과 즉시성에 강하다. 과제 제출처럼 구조를 설명하고 비교 분석까지 해야 하는 상황에서는 Make가 더 설득력 있는 화면과 흐름을 제공하였고, 실제 실무처럼 품 제출 직후 자동 반응을 보여 주는 데에는 Zapier가 더 자연스러웠다. 따라서 두 도구는 우열 관계라기보다, 자동화의 목적과 강조점에 따라 선택해야 하는 상호보완적 플랫폼이라고 결론지을 수 있다. [Source Source Source Source](#)

결론

본 프로젝트를 통해 동일한 자동화 과제를 [Make](#)와 [Zapier](#)에서 각각 구현해 봄으로써, 노코드 자동화 도구의 공통 구조와 차별적 특징을 동시에 확인할 수 있었다. 두 도구 모두 Google Form 응답을 기반으로 조건 분기, 결과 기록, 메신저 알림 전송을 구현할 수 있었으며, 과제의 핵심 요구사항인 Trigger, Action, 조건 분기를 모두 충족하였다. [Source Source](#)

Make는 Router를 중심으로 워크플로우 구조를 명확하게 시각화할 수 있어 설계 흐름을 설명하기에 적합하였고, Zapier는 Google Forms의 즉시 트리거와 단계형 편집기를 바탕으로 더 빠르고 직관적인 자동화 구성이 가능하였다. 따라서 학습 및 보고서 작성 목적에서는 Make가, 빠른 실무 적용과 즉시성 중심의 자동화에서는 Zapier가 더 적합하다고 판단하였다. [Source Source](#)

최종적으로 본 과제는 단순히 도구 사용법을 익히는 수준을 넘어, 동일한 업무를 서로 다른 플랫폼에서 자동화하면서 도구별 설계 철학과 실행 방식의 차이를 이해하는 데 의미가 있었다. 이러한 경험은 향후

자동화 도구를 선택할 때 단순 기능 비교가 아니라, 업무 목적, 반응 속도, 설명 가능성, 운영 비용 등 다양한 관점을 함께 고려해야 함을 보여 준다

